

Tribhuvan University
IOST
Prithvi Narayan Campus



Lab Report of Compiler Design and Construction by:

Vivek Paudel

Sixth Semester

Section: A

Roll no: 80012185 (24)

Submitted To:

Tara Bahadur Thapa

(CDC Teacher)

INDEX

Name: Vivek Paudel

Roll No: 80012185 (24)

SN	Name of Experiment	Date of submission	Signature
1	WAP to check whether user given string is valid comment or not		
2	Write a C program to test whether a given identifier is valid or not		
3	WAP to recognize string under "a*, a*b+, abb"		
4	WAP to recognize the string (a+b)+		
5	WAP for Lexical Analyzer in C		
6	WAP that finds the FIRST of a given grammar		
7	WAP to find the Follow		
8	Write a C program for constructing of LL (1) parsing		
9	Write a C Program to Implement Shift Reduce Parser		
10	Write a C-program for intermediate Code Generation		
11	Write a C-program for Final Code Generation		

Experiment 1: WAP to check whether user given string is valid comment or not

Theory

In compiler construction, the lexical analysis phase serves as the initial front-end stage responsible for scanning the raw input source code character by character and converting it into a sequence of meaningful tokens. During this process, non-executable elements such as comments and extraneous whitespace must be stripped away to streamline syntax analysis and reduce the memory footprint of downstream parsing phases.

C programming supports two distinct forms of comment syntax:

- **Single-line comments:** Initiated by the two-character sequence `//` and terminating at the end-of-line newline character (`\n`).
- **Multi-line comments:** Initiated by `/*` and terminated by `*/`. Multi-line comments can span across multiple lines and must contain a closing pair; an missing `*/` results in a lexical error.

To implement a scanner for comment recognition, the string is evaluated using deterministic state transitions. The lexical scanner checks if the first character is `/`. If the subsequent character is `/`, the string is immediately validated as a single-line comment. If the second character is `*`, the scanner enters a search state scanning subsequent indices for the closing sequence `*/`. If found prior to string termination, it is classified as a valid multi-line comment. Any violation of these delimiter structures causes the string to be rejected as an invalid comment.

Source Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char com[100];
    int i, is_multiline_valid = 0;

    printf("Enter comment string: ");
    fgets(com, sizeof(com), stdin);
    com[strcspn(com, "\n")] = 0; // Strip newline

    int len = strlen(com);

    if (len >= 2 && com[0] == '/') {
        if (com[1] == '/') {
            printf("Result: Valid Single-Line Comment\n");
            return 0;
        }
    }
```

```

    } else if (com[1] == '*') {
        for (i = 2; i < len - 1; i++) {
            if (com[i] == '*' && com[i + 1] == '/') {
                is_multiline_valid = 1;
                break;
            }
        }
        if (is_multiline_valid) {
            printf("Result: Valid Multi-Line Comment\n");
        } else {
            printf("Result: Invalid Multi-Line Comment (Unterminated '*/')\n");
        }
        return 0;
    }
}

printf("Result: Not a valid comment\n");
return 0;
}

```

Output

Output

```

Enter comment string: Vivek does not comment
Result: Not a valid comment

```

Output

```

Enter comment string: /* vivek comments now */
Result: Valid Multi-Line Comment

```

Conclusion

The experiment successfully demonstrates the state pattern matching required by lexical analyzers to detect and isolate comments. By implementing string-scanning algorithms for `//` and `/* ... */`, the program illustrates how lexical scanners pre-process source code to eliminate non-functional tokens before passing the stream to the syntax analyzer.

Experiment 2: Write a C program to test whether a given identifier is valid or not

Theory

Identifiers are user-defined names given to program entities like variables, functions, arrays, and structures. The lexical analyzer validates identifiers using predefined formal rules dictated by the language grammar specification. In the standard C language specification, a valid identifier must conform to the regular expression grammar:

Identifier $\rightarrow [a - zA - Z_][a - zA - Z0 - 9_]*$

The rules for valid identifiers in C state that:

1. The first character must strictly be an alphabetic letter (uppercase or lowercase) or an underscore (_). It cannot begin with a digit or special character.
2. Subsequent characters may consist of alphabetic letters, numeric digits (0-9), or underscores.
3. Special characters (such as @, \$, %, -) and spaces are strictly prohibited within an identifier token.
4. The identifier name must not match any reserved language keywords (such as int, return, void, if, while).

The program simulates this validation by combining regular character set checks using `isalpha()`, `isdigit()`, and `isalnum()` with string comparison against a lookup array of standard C keywords.

Source Code:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int isKeyword(char str[]) {
    char keywords[32][10] = {
        "auto", "break", "case", "char", "const", "continue", "default", "do",
        "double", "else", "enum", "extern", "float", "for", "goto", "if",
        "int", "long", "register", "return", "short", "signed", "sizeof", "static",
        "struct", "switch", "typedef", "union", "unsigned", "void", "volatile", "while"
    };
    for (int i = 0; i < 32; i++) {
        if (strcmp(keywords[i], str) == 0) {
            return 1;
        }
    }
    return 0;}

int main() {
    char str[50];
```

```

int flag = 1;

printf("Enter an identifier: ");
scanf("%s", str);

if (isKeyword(str)) {
    printf("Result: Invalid Identifier (Reserved Keyword)\n");
    return 0;
}

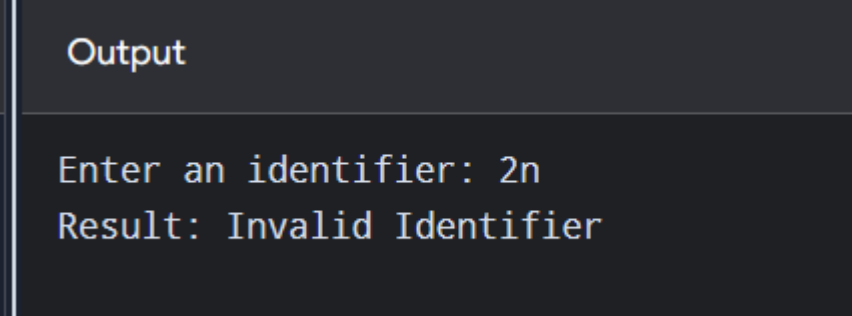
if (isalpha(str[0]) || str[0] == '_') {
    for (int i = 1; i < strlen(str); i++) {
        if (!isalnum(str[i]) && str[i] != '_') {
            flag = 0;
            break;
        } } } else {
    flag = 0;
}

if (flag) {
    printf("Result: Valid Identifier\n");
} else {
    printf("Result: Invalid Identifier\n");
}

return 0;
}

```

Output



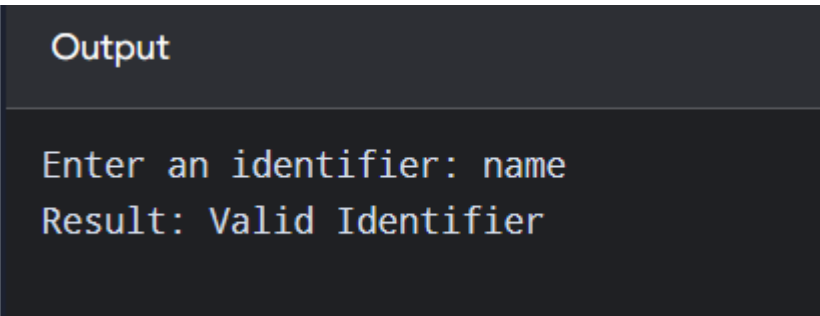
A terminal window with a dark background. The title bar says "Output". The prompt "Enter an identifier: " is followed by the input "2". The output is "Result: Invalid Identifier".

```

Output

Enter an identifier: 2
Result: Invalid Identifier

```



A terminal window with a dark background. The title bar says "Output". The prompt "Enter an identifier: " is followed by the input "name". The output is "Result: Valid Identifier".

```

Output

Enter an identifier: name
Result: Valid Identifier

```

Conclusion

This experiment highlights the lexical token validation process for identifiers. Checking character types and comparing against reserved keyword sets ensures that user-defined tokens adhere to language rules before being indexed into the compiler's symbol table.

Experiment 3: WAP to recognize string under "a*, a*b+, abb"

Theory

Formal languages and regular expressions form the foundation of lexical analyzer design. Regular expressions describe set patterns that can be processed using Deterministic Finite Automata (DFA). This experiment recognizes strings belonging to three specific regular expressions over the alphabet $\Sigma = \{a, b\}$

1. **a***: Represents the Kleene closure of 'a', matching zero or more occurrences of 'a' (e.g., ϵ , a, aa, aaa).
2. **a*b+**: Matches zero or more 'a's followed strictly by one or more 'b's (e.g., b, ab, abb, aab, bb).
3. **abb**: Represents an explicit string literal matching only abb.

In a compiler, when multiple regular expressions overlap (such as abb matching both a*b+ and abb), pattern matching rules such as explicit matching priority or longest match selection determine token assignment. The program evaluates character sequences linearly to determine which formal pattern the input belongs to.

Source Code

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int i = 0;

    printf("Enter string over alphabet {a, b}: ");
    scanf("%s", str);

    if (strcmp(str, "abb") == 0) {
        printf("Result: String matched pattern 'abb'\n");
        return 0;
    }

    while (str[i] == 'a') {
        i++;
    }

    if (str[i] == '\0') {
        printf("Result: String matched pattern 'a*'\n");
        return 0;
    }

    if (str[i] == 'b') {
```

```
while (str[i] == 'b') {  
    i++;  
}  
if (str[i] == '\0') {  
    printf("Result: String matched pattern 'a*b+\n");  
    return 0;  
}  
}  
  
printf("Result: String does NOT belong to any given patterns\n");  
return 0;  
}
```

Output

Output

```
Enter string over alphabet {a, b}: aaabb  
Result: String matched pattern 'a*b+'
```

Output

```
Enter string over alphabet {a, b}: aaaa  
Result: String matched pattern 'a*'
```

Output

```
Enter string over alphabet {a, b}: baa  
Result: String does NOT belong to any given patterns
```

Conclusion

The algorithm illustrates finite state transitions and pattern prioritization in token recognition. Processing input characters through conditional loops mirrors how a scanner transitions between DFA states to classify string inputs into formal language categories.

Experiment 4: WAP to recognize the string (a+b)+

Theory

The regular expression (a+b)+ specifies a regular language over the alphabet $\Sigma = \{a, b\}$. The positive closure operator (+) requires that any valid string must consist of one or more symbols drawn from the set $\{a, b\}$.

Formal criteria for language membership:

1. Non-emptiness: The length of the string must be greater than or equal to 1. The empty string (epsilon) is rejected.
2. Alphabet constraint: Every individual character within the string must strictly belong to the set $\{a, b\}$. Any appearance of other symbols (e.g., 'c', '0', spaces) causes immediate rejection.

A simple Deterministic Finite Automaton (DFA) for (a+b)+ consists of two states:

- State q0 (Start): Unaccepted initial state. On reading 'a' or 'b', transitions to state q1.
- State q1 (Accepting): On reading 'a' or 'b', loops back to q1. On encountering any symbol outside $\{a, b\}$, transitions to a dead/trap state.

Code

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int i, valid = 1;

    printf("Enter string: ");
    scanf("%s", str);

    int len = strlen(str);

    if (len == 0) {
        valid = 0;
    } else {
        for (i = 0; i < len; i++) {
            if (str[i] != 'a' && str[i] != 'b') {
                valid = 0;
                break;
            }
        }
    }
}
```

```
if (valid) {  
    printf("Result: String accepted under pattern (a+b)+\n");  
} else {  
    printf("Result: String REJECTED under pattern (a+b)+\n");  
}  
  
return 0;  
}
```

Output

Output

```
Enter string: ababaab  
Result: String accepted under pattern (a+b)+
```

Output

```
Enter string: cc  
Result: String REJECTED under pattern (a+b)+
```

Conclusion

The experiment verifies positive closure behavior in regular expressions. It demonstrates state validation mechanics where non-empty requirements and alphabet restrictions are enforced programmatically.

Experiment 5: WAP for Lexical Analyzer in C

Theory

A Lexical Analyzer (or scanner) converts high-level source code character streams into structured token representations. It forms the foundation of the compiler front-end, performing tasks such as character buffering, lexeme identification, and symbol table registration.

Tokens processed during lexical analysis are categorized into:

- Keywords: Language-defined reserved words (int, if, while).
- Identifiers: User-defined symbol names (count, total).
- Operators: Arithmetic or logical operation symbols (+, -, *, =, ==).
- Constants: Literal values (numerical integer or floating-point values).
- Delimiters: Structural tokens (;, {, }).

The lexical analyzer uses a character buffer to read source code, skips whitespace like spaces and newlines, identifies operators immediately, and accumulates alphanumeric characters to check against keyword lookup tables before classifying them as identifiers or constants.

Source Code ---> [Lexical Analyzer] ---> Token Stream ---> [Syntax Analyzer]

Code

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int isKeyword(char buf[]) {
    char keywords[5][10] = {"int", "float", "if", "else", "while"};
    for (int i = 0; i < 5; i++) {
        if (strcmp(keywords[i], buf) == 0) return 1;
    }
    return 0;
}

int main() {
    char input[] = "int x = y + 25 ;";
    char ch, buf[20], operators[] = "+-*/=";
    int j = 0, len = strlen(input);

    printf("Source Code: %s\n\nToken Breakdown:\n", input);

    for (int i = 0; i < len; i++) {
```

```

ch = input[i];

// Check for operators
for (int k = 0; k < 5; k++) {
    if (ch == operators[k]) {
        printf("Operator : %c\n", ch);
    }
}

// Accumulate alphanumeric lexemes
if (isalnum(ch)) {
    buf[j++] = ch;
} else if ((ch == ' ' || ch == '\n' || ch == '\t' || ch == ';') && (j != 0)) {
    buf[j] = '\0';
    j = 0;

    if (isKeyword(buf)) {
        printf("Keyword : %s\n", buf);
    } else if (isdigit(buf[0])) {
        printf("Constant : %s\n", buf);
    } else {
        printf("Identifier: %s\n", buf);
    }
}

if (ch == ';') {
    printf("Delimiter : ;\n");
}

return 0;
}

```

Output

Output

Source Code: `int x = y + 25 ;`

Token Breakdown:

Keyword : `int`

Identifier: `x`

Operator : `=`

Identifier: `y`

Operator : `+`

Constant : `25`

Delimiter : `;`

Conclusion

This experiment implements a functional lexical scanner in C. It demonstrates how compilers parse stream inputs into distinct lexical categories, paving the way for grammar processing in the syntax analysis phase.

Experiment 6: WAP that finds the FIRST of a given grammar

Theory

In top-down parsing (such as LL(1) predictive parsing), determining the FIRST set for grammar non-terminals is necessary to construct non-backtracking parse tables. FIRST(X) is defined as the set of terminal symbols that appear as the first symbol in any string derived from X. If X derives epsilon, then epsilon is also included in FIRST(X).

Rules for calculating FIRST:

1. If X is a terminal, then $\text{FIRST}(X) = \{X\}$.
2. If $X \rightarrow \epsilon$ is a production, then add epsilon to FIRST(X).
3. If X is a non-terminal and $X \rightarrow Y_1 Y_2 \dots Y_k$, then:
 - Add FIRST(Y₁) (excluding epsilon) to FIRST(X).
 - If FIRST(Y₁) contains epsilon, add FIRST(Y₂) (excluding epsilon), and continue sequentially.
 - If all Y₁ through Y_k derive epsilon, add epsilon to FIRST(X).

Computing FIRST enables a top-down parser to look at the next incoming input token and deterministically select the correct grammar production rule.

Code

```
#include <stdio.h>
#include <ctype.h>

int num_productions = 3;
char productionSet[10][10] = {"S=aAB", "A=b", "B=c"};

void findFirst(char c) {
    for (int i = 0; i < num_productions; i++) {
        if (productionSet[i][0] == c) {
            char target = productionSet[i][2];
            if (islower(target) || target == 'a' || target == 'b' || target == 'c') {
                printf("%c ", target);
            } else {
                findFirst(target);
            }
        }
    }
}

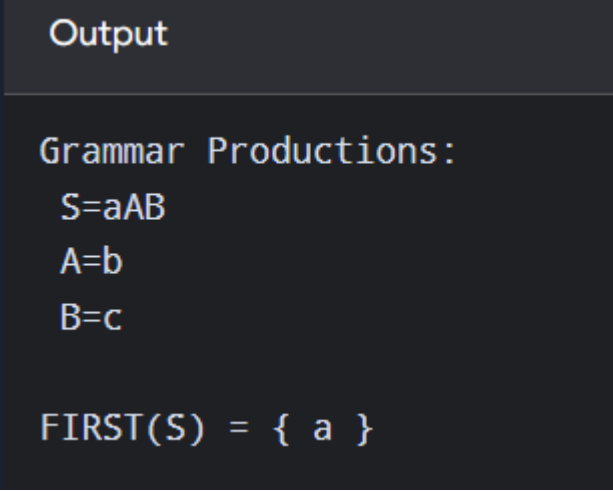
int main() {
```

```
char symbol = 'S';
printf("Grammar Productions:\n");
for (int i = 0; i < num_productions; i++) {
    printf(" %s\n", productionSet[i]);
}

printf("\nFIRST(%c) = { ", symbol);
findFirst(symbol);
printf("}\n");

return 0;
}
```

Output



```
Output

Grammar Productions:
S=aAB
A=b
B=c

FIRST(S) = { a }
```

Conclusion

The program recursively evaluates grammar production heads to compute the FIRST set. Computing these sets is essential for resolving non-determinism during predictive parsing table construction.

Experiment 7: WAP to find the Follow

Theory

While the FIRST set looks forward at what symbols a non-terminal can generate, the FOLLOW set examines what terminals can immediately succeed a non-terminal in any sentential form derived from the start symbol. FOLLOW(A) is used in top-down parsing to handle epsilon-productions and construct LL(1) parse tables.

Rules for calculating FOLLOW(A):

1. For the start symbol S, place the end-of-input marker '\$' into FOLLOW(S).
2. If there is a production rule $A \rightarrow \alpha B \beta$, then everything in FIRST(β) (except epsilon) is in FOLLOW(B).
3. If a production $A \rightarrow \alpha B$ exists, or $A \rightarrow \alpha B \beta$ exists where FIRST(β) contains epsilon, then everything in FOLLOW(A) is included in FOLLOW(B).

Definition: $\text{FOLLOW}(A) = \{ a \text{ in Terminals} \mid S \text{ derives } \alpha A a \beta \}$

By knowing what tokens can legally follow a non-terminal A, the parser can safely choose an epsilon-expansion when A encounters a token in FOLLOW(A).

Code

```
#include <stdio.h>
#include <string.h>

int n = 3;
char grammar[10][10] = {"S=AB", "A=a", "B=b"};

void findFollow(char c) {
    if (grammar[0][0] == c) {
        printf("$ ");
    }

    for (int i = 0; i < n; i++) {
        for (int j = 2; j < strlen(grammar[i]); j++) {
            if (grammar[i][j] == c) {
                if (grammar[i][j + 1] != '\0') {
                    printf("%c ", grammar[i][j + 1]);
                } else if (grammar[i][0] != c) {
                    findFollow(grammar[i][0]);
                }
            }
        }
    }
}
```

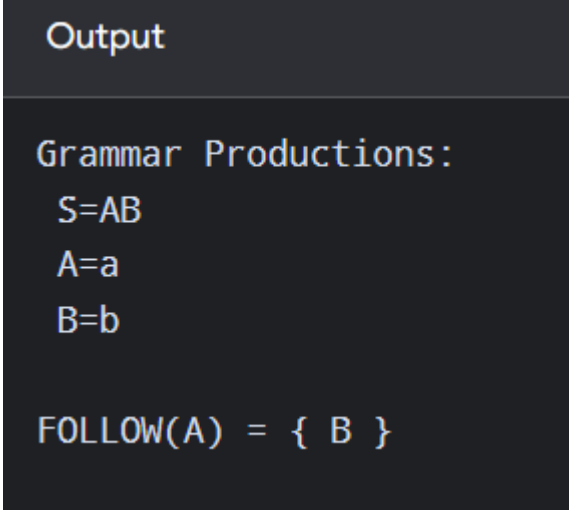
```
int main() {
    char symbol = 'A';

    printf("Grammar Productions:\n");
    for (int i = 0; i < n; i++) {
        printf(" %s\n", grammar[i]);
    }

    printf("\nFOLLOW(%c) = { ", symbol);
    findFollow(symbol);
    printf("}\n");

    return 0;
}
```

Output



```
Output

Grammar Productions:
S=AB
A=a
B=b

FOLLOW(A) = { B }
```

Conclusion

The experiment successfully computes the FOLLOW set for grammar non-terminals. Combining FIRST and FOLLOW calculations provides the mathematical framework required for LL(1) parse table construction.

Experiment 8: Write a C program for constructing of LL (1) parsing

Theory

An LL(1) parser is a deterministic, top-down predictive parser. The name LL(1) signifies:

- L: Left-to-right scanning of input stream.
- L: Leftmost derivation generation.
- 1: One single lookahead token used to make parsing decisions.

To build an LL(1) parser, a parsing table $M[A, a]$ is generated, where A represents a non-terminal and 'a' represents a terminal or the end marker '\$'.

Construction rules for Table $M[A, a]$:

1. For each production $A \rightarrow \alpha$:
 - For each terminal 'a' in $FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$.
 - If epsilon is in $FIRST(\alpha)$, for each terminal 'b' in $FOLLOW(A)$ (including '\$'), add $A \rightarrow \alpha$ to $M[A, b]$.
2. All undefined cells in $M[A, a]$ are flagged as Error entries.

If any cell $M[A, a]$ receives multiple production rules, the grammar is not LL(1) because it contains conflicts (such as $FIRST/FIRST$ or $FIRST/FOLLOW$ conflicts).

Code

```
#include <stdio.h>
#include <string.h>

int main() {
    char nonTerminals[] = {'S', 'A'};
    char terminals[] = {'a', 'b', '$'};
    char table[2][3][10];

    // Initialize table with "Error"
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            strcpy(table[i][j], "Error");
        }
    }
}
```

```

// Populate LL(1) parse rules
// S -> aA for lookahead 'a'
strcpy(table[0][0], "S->aA");
// A -> b for lookahead 'b'
strcpy(table[1][1], "A->b");

printf("LL(1) Predictive Parsing Table:\n");
printf("-----\n");
printf("\t");
for (int j = 0; j < 3; j++) {
    printf("%c\t", terminals[j]);
}
printf("\n-----\n");

for (int i = 0; i < 2; i++) {
    printf("%c |", nonTerminals[i]);
    for (int j = 0; j < 3; j++) {
        printf("%s\t", table[i][j]);
    }
    printf("\n");
}
printf("-----\n");

return 0;
}

```

Output

Output			
LL(1) Predictive Parsing Table:			

	a	b	\$

S	S->aA	Error	Error
A	Error	A->b	Error

Conclusion

The program builds a structured representation of an LL(1) parsing table. Predictive parsing tables eliminate backtracking, enabling efficient deterministic $O(n)$ syntax analysis.

Experiment 9: Write a C Program to Implement Shift Reduce Parser

Theory

Shift-Reduce parsing is a bottom-up parsing strategy that builds a parse tree starting from the input leaf terminals up to the grammar start symbol. The parser maintains an operational stack and an input buffer.

The parser executes four primary actions:

1. Shift: Reads the current input token and pushes it onto the parse stack.
2. Reduce: Replaces a stack substring (matching the right-hand side of a production rule, called a handle) with the corresponding left-hand non-terminal.
3. Accept: Declares successful parse completion when the stack contains only the start symbol and the input buffer is empty.
4. Error: Occurs when no valid shift or reduce step can be taken.

Stack Parsing State:

Stack: \$Input Buffer: a\$ Action: Shift

Stack: \$a Input Buffer:\$ Action: Reduce E -> a

Stack: \$E Input Buffer:\$ Action: Accept

Bottom-up parsers must avoid Shift/Reduce or Reduce/Reduce conflicts where the parser cannot deterministically decide whether to shift another token or reduce the current stack contents.

Code

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[16], stack[16];
    int stack_ptr = 0, input_ptr = 0;

    printf("Grammar: E -> a\n");
    printf("Enter input string: ");
    scanf("%s", input);

    int len = strlen(input);
    stack[0] = '\0';

    printf("\nSTACK\t\tINPUT\t\tACTION\n");
    printf("-----\n");
```

```

while (input_ptr < len) {
    // Shift operation
    stack[stack_ptr] = input[input_ptr];
    stack[stack_ptr + 1] = '\0';
    input[input_ptr] = ' ';
    input_ptr++;
    stack_ptr++;

    printf("%%-10s\t%-10s$\tShift\n", stack, input + input_ptr);

    // Reduce operation check (E -> a)
    if (stack[stack_ptr - 1] == 'a') {
        stack[stack_ptr - 1] = 'E';
        printf("%%-10s\t%-10s$\tReduce E -> a\n", stack, input + input_ptr);
    }
}

if (stack[0] == 'E' && stack[1] == '\0') {
    printf("-----\n");
    printf("Result: String ACCEPTED (Successful Reduction)\n");
} else {
    printf("-----\n");
    printf("Result: String REJECTED\n");
}

return 0;
}

```

Output

```

Output

Grammar: E -> a
Enter input string: a

STACK      INPUT      ACTION
-----
$a          $ Shift
$E          $ Reduce E -> a
-----
Result: String ACCEPTED (Successful Reduction)

```

Conclusion

The experiment models the fundamental mechanics of stack-based bottom-up parsing. Shift-Reduce parsing forms the foundation for LR parsers, including LALR parsers used in compiler construction tools like YACC and Bison.

Experiment 10: Write a C-program for intermediate Code Generation

Theory

Intermediate Code Generation acts as a bridge between the front-end (lexical, syntax, and semantic analysis) and back-end phases (target machine optimization and code generation) of a compiler.

[Front-End Phase] ---> Intermediate Representation (IR) ---> [Back-End Phase]

Key benefits of Intermediate Representation (IR):

- Portability: Adapting a compiler to a new architecture requires updating only the back-end while keeping the front-end intact.
- Optimization: Machine-independent code optimizations (such as dead code elimination and constant folding) are performed directly on IR.

A common IR format is Three-Address Code (TAC). In TAC, every instruction contains at most three memory addresses/operands and one operator:

$x = y \text{ op } z$

Complex arithmetic expressions are linearized using temporary variables (t1, t2, etc.) to break multi-operator expressions into sequential instructions.

Code

```
#include <stdio.h>
#include <string.h>

int main() {
    char expr[20];
    char arg1, op, arg2;
    char tempVar = 't';
    int tempCount = 1;

    printf("Enter simple assignment expression (e.g., X=A+B): ");
    scanf("%s", expr);

    printf("\nIntermediate Code (Three-Address Code):\n");
    printf("-----\n");

    // Parsing format: Result = Arg1 Op Arg2
    if (strlen(expr) >= 5 && expr[1] == '=') {
        arg1 = expr[2];
```

```

    op = expr[3];
    arg2 = expr[4];

    // Generate temporary evaluation instruction
    printf("%c%d = %c %c %c\n", tempVar, tempCount, arg1, op, arg2);
    // Final assignment instruction
    printf("%c = %c%d\n", expr[0], tempVar, tempCount);
} else {
    printf("Invalid input expression format.\n");
}

return 0;
}

```

Output

```

Output

Enter simple assignment expression (e.g., X=A+B): J=0+T

Intermediate Code (Three-Address Code):
-----
t1 = 0 + T
J  = t1

```

Conclusion

The experiment demonstrates how high-level binary arithmetic expressions are broken down into sequential Three-Address Code instructions, laying the groundwork for machine-independent code optimization.

Experiment 11: Write a C-program for Final Code Generation

Theory

Final Code Generation is the final phase of compilation. It translates intermediate code (like Three-Address Code) into target machine instructions or assembly code for a specific processor architecture.

Main tasks in final code generation:

1. Instruction Selection: Choosing target machine instructions that efficiently implement each IR statement.
2. Register Allocation: Assigning temporary variables and identifiers to physical CPU registers (R0, R1, etc.) to minimize memory access latency.
3. Instruction Ordering: Ordering assembly instructions to maximize instruction-level parallelism and hardware execution efficiency.

In this experiment, TAC statements are translated into hypothetical assembly language instructions using an accumulator register (R0):

- Expression $x = y \text{ op } z$ translates to:
 1. MOV R0, y (Load operand y into accumulator register R0)
 2. ADD/SUB/MUL/DIV R0, z (Perform operation with operand z)
 3. MOV x, R0 (Store result back to memory variable x)

Code

```
#include <stdio.h>

int main() {
    char op, arg1, arg2, result;

    printf("Enter Intermediate Code Statement: ");
    scanf(" %c=%c%c%c", &result, &arg1, &op, &arg2);

    printf("\nGenerated Machine Assembly Code:\n");
    printf("-----\n");
    printf("MOV R0, %c\n", arg1);

    switch (op) {
        case '+':
            printf("ADD R0, %c\n", arg2);
            break;
```

```

    case '-':
        printf("SUB R0, %c\n", arg2);
        break;
    case '*':
        printf("MUL R0, %c\n", arg2);
        break;
    case '/':
        printf("DIV R0, %c\n", arg2);
        break;
    default:
        printf("; Unknown Operator\n");
        return 1;
}

printf("MOV %c, R0\n", result);
printf("-----\n");

return 0;
}

```

Output

```

Output

Enter Intermediate Code Statement: Z=A+B

Generated Machine Assembly Code:
-----
MOV R0, A
ADD R0, B
MOV Z, R0
-----

```

Conclusion

The experiment illustrates how compilers translate abstract intermediate representations into target assembly instructions. This highlights the role of register allocation and instruction mapping during final code generation.